

Transformer From Scratch - Detailed Beginner Textbook

This textbook explains the uploaded Transformer notebook step-by-step. It follows the flow from text preprocessing, vocabulary creation, embeddings, positional encoding, attention masks, multi-head attention, encoder layers, decoder layers, residual connections, feed-forward networks, and final predictions. The goal is to explain not only what each function does but also how vectors move through the network using concrete examples.

Chapter 1: Vocabulary

Example vocabulary: =0, =1, =2, [sep]=3 Sentence: 'anugrah is learning' After adding special tokens: anugrah is learning [sep] Token IDs might become: [2,1,5,9,3] The neural network only sees numbers, not text.

Chapter 2: Padding and Attention Masks

Suppose max sequence length = 10. Before padding: [2,1,5,9,3] After padding: [2,1,5,9,3,0,0,0,0,0] Attention mask: [1,1,1,1,1,0,0,0,0,0] 1 means keep token. 0 means ignore token.

Chapter 3: Embeddings

The notebook uses $d_{\text{model}} = 8$. Each token becomes an 8-dimensional vector. Example: learning $\rightarrow [0.3, -0.2, 0.5, 0.1, -0.4, 0.7, 0.2, 0.9]$ Shape: (batch_size, sequence_length, 8)

Chapter 4: Positional Encoding

Transformers do not naturally know token order. Sinusoidal encoding is added:
 $PE(pos, 2i) = \sin(pos / (10000^{(2i/d_{\text{model}})}))$
 $PE(pos, 2i+1) = \cos(pos / (10000^{(2i/d_{\text{model}})}))$
Final input: Embedding + Positional Encoding

Chapter 5: Q, K, V

Input vector: $x = [1, 2, 3, 4]$ Assume: $W_q = \text{Identity}$ $W_k = \text{Identity}$ $W_v = \text{Identity}$ Then: $Q = [1, 2, 3, 4]$
 $K = [1, 2, 3, 4]$ $V = [1, 2, 3, 4]$ In practice W_q, W_k, W_v are learned matrices.

Chapter 6: Attention Score Calculation

Attention score: $\text{Score} = Q \times K^T / \sqrt{\text{head_dim}}$ Example: $Q = [1, 2]$ $K = [3, 4]$ $QK^T = (1 \times 3) + (2 \times 4) = 11$ $\text{head_dim} = 2$ $\text{score} = 11 / \sqrt{2} = 7.78$ Softmax converts scores into probabilities.

Chapter 7: Masking

Code: `scores = scores.masked_fill(mask==0, -inf)` Example: `scores=[2.5,1.8,3.2,0.9]` `mask=[1,1,0,0]` After masking: `[2.5,1.8,-inf,-inf]` Softmax: `[0.668,0.332,0,0]` Padding tokens receive zero attention.

Chapter 8: Multi Head Attention

$d_{\text{model}} = 8$ $\text{num_heads} = 2$ Input: `[1,2,3,4,5,6,7,8]` Split: `Head1=[1,2,3,4]` `Head2=[5,6,7,8]`
Attention runs independently. Outputs are concatenated back together. Result: `[1,2,3,4,5,6,7,8]`

Chapter 9: Residual Connections

The notebook uses: $x + \text{attention_output}$ Example: $x = [1, 2, 3]$ $\text{attention} = [0.1, 0.5, -0.2]$ Result: `[1.1,2.5,2.8]` This helps gradient flow.

Chapter 10: Feed Forward Network

Structure: Linear $\rightarrow$ ReLU $\rightarrow$ Linear Example: Input: `[2,-1,3]` After ReLU: `[2,0,3]` Allows non-linear learning.

Chapter 11: Encoder Layer Flow

Input -> QKV -> Multi Head Attention -> Dropout -> Residual -> LayerNorm -> Feed Forward -> Residual -> LayerNorm Output becomes encoder representation.

Chapter 12: Decoder Layer Flow

Target Tokens -> Masked Self Attention -> Residual + Norm -> Encoder Decoder Attention -> Residual + Norm -> Feed Forward -> Residual + Norm Cross attention uses: Q from decoder K from encoder V from encoder

Chapter 13: Full Transformer

Source Sentence -> Encoder Target Sentence -> Decoder Encoder output is consumed by decoder cross-attention. Final linear layer projects: (batch,seq,d_model) to (batch,seq,vocab_size) which predicts next words.

Notebook Code Cells Included

Code Cell 1

```
import torch import torch.nn as nn import math
```

Code Cell 2

```
# 1. Define the Vocabulary (Adding '[sep]') new_vocab = {'<pad>': 0,
'<unk>': 1, '<cls>': 2, '[sep]': 3, 'this': 4, 'is': 5, 'a': 6,
'positive': 7, 'sentence': 8, 'learning': 9, 'about': 10, 'transformers':
11, 'fun': 12, 'i': 13, 'hate': 14, 'am': 15, 'so': 16, 'disappointed':
17, 'really': 18, 'enjoyed': 19, 'movie': 20, 'wonderful': 21, 'day': 22,
'for': 23, 'walk': 24, 'product': 25, 'excellent!': 26, 'absolutely': 27,
'dreadful': 28, 'service': 29, 'what': 30, 'terrible': 31, 'experience':
32, 'feeling': 33, 'very': 34, 'happy': 35, 'today': 36, 'boy': 37,
'anugrah': 38} # 2. Define the Reverse Vocabulary idx_to_word = {i: word
for word, i in new_vocab.items()} vocab_size = len(new_vocab)
print(f"Vocabulary size is: {vocab_size}")
```

Code Cell 4

```
# Helper to add special tokens and initial segment IDs def
_add_special_tokens_and_segments(input_text, vocab): tokens =
[vocab.get("<cls>", "<cls>")] # Start with <cls> segment_ids = [0] #
<cls> belongs to segment 0 # Tokenize input text and add words for word
in input_text.lower().replace('.', '').split(): tokens.append(word)
segment_ids.append(0) # All words in the first sentence belong to segment
0 tokens.append(vocab.get("[sep]", "[sep]")) # Add [sep]
segment_ids.append(0) # [sep] belongs to segment 0 return tokens,
segment_ids # Helper to convert tokens to IDs def _convert_to_ids(tokens,
vocab): return [vocab.get(token, vocab["<unk>"]) for token in tokens] #
Helper to pad token IDs and segment IDs def
_pad_ids_and_segments(token_ids, segment_ids, max_seq_len, pad_token_id):
if len(token_ids) > max_seq_len: token_ids = token_ids[:max_seq_len]
segment_ids = segment_ids[:max_seq_len] else: padding_length =
max_seq_len - len(token_ids) token_ids.extend([pad_token_id] *
padding_length) segment_ids.extend([0] * padding_length) # Padding tokens
belong to segment 0 return token_ids, segment_ids # Helper to create
attention mask def _create_attention_mask_from_ids(token_ids,
pad_token_id): mask = [1 if token_id != pad_token_id else 0 for token_id
in token_ids] return torch.tensor(mask,
dtype=torch.long).unsqueeze(0).unsqueeze(1).unsqueeze(2) # [batch_size,
1, 1, seq
```

Code Cell 5

```
def prepare_transformer_inputs(input_text, vocab, max_seq_len):
pad_token_id = vocab['<pad>'] # Step 1: Add special tokens and assign
initial segment IDs tokens, segment_ids =
_add_special_tokens_and_segments(input_text, vocab) # Step 2: Convert
tokens to IDs token_ids = _convert_to_ids(tokens, vocab) # Step 3: Pad
token IDs and segment IDs padded_token_ids, padded_segment_ids =
_pad_ids_and_segments(token_ids, segment_ids, max_seq_len, pad_token_id
) # Convert to PyTorch tensors and add batch dimension input_ids =
torch.tensor(padded_token_ids, dtype=torch.long).unsqueeze(0)
```

```
token_type_ids = torch.tensor(padded_segment_ids,
dtype=torch.long).unsqueeze(0) # Step 4: Create attention mask
attention_mask = _create_attention_mask_from_ids(padded_token_ids,
pad_token_id) return input_ids, attention_mask, token_type_ids
```

Code Cell 7

```
input_text = "anugrah is learning" max_seq_len = 10 # maximum token need
to be created # Prepare inputs using the new modular function input_ids,
attention_mask_new, token_type_ids_new = prepare_transformer_inputs(
input_text, new_vocab, max_seq_len ) print(f"Input Text: '{input_text}'")
print(f"Input IDs: {input_ids}") print(f"Attention Mask:
{attention_mask_new}") print(f"Token Type IDs: {token_type_ids_new}")
print(f"Input IDs shape: {input_ids.shape}") print(f"Attention Mask
shape: {attention_mask_new.shape}") print(f"Token Type IDs shape:
{token_type_ids_new.shape}")
```

Code Cell 9

```
d_model = 8 #vector dimension for each word embedding = nn.Embedding(
num_embeddings=vocab_size, embedding_dim=d_model ) embedded_input =
embedding(input_ids) print(f"Shape of embedded input:
{embedded_input.shape}")
```

Code Cell 10

```
print(embedded_input)
```

Code Cell 11

```
print(f"Input Text: '{input_text}'") print(f"Input IDs:
{input_ids.squeeze(0)}") print("\nEncodings for each token in 'anugrah is
learning':") # Iterate through the input_ids and corresponding embedded
vectors for i, token_id in enumerate(input_ids.squeeze(0)): word =
idx_to_word.get(token_id.item(), '<unknown>') embedding_vector =
embedded_input.squeeze(0)[i] print(f"Token: '{word}' (ID:
{token_id.item()}) -> Embedding: {embedding_vector.tolist()}")
```

Code Cell 12

```
class PositionalEncoding(nn.Module): def __init__(self, d_model,
max_seq_len=5000): super().__init__() self.dropout = nn.Dropout(0.1) #
Create a positional encoding matrix pe = torch.zeros(max_seq_len,
d_model) position = torch.arange(0, max_seq_len,
dtype=torch.float).unsqueeze(1) div_term = torch.exp(torch.arange(0,
d_model, 2).float() * (-math.log(10000.0) / d_model)) pe[:, 0::2] =
torch.sin(position * div_term) pe[:, 1::2] = torch.cos(position *
div_term) if d_model % 2 == 0 else torch.cos(position * div_term[:-1]) pe
= pe.unsqueeze(0) # Add batch dimension self.register_buffer('pe', pe)
def forward(self, x): # Add positional encoding to the input embeddings #
x has shape (batch_size, seq_len, d_model) x = x + self.pe[:, :x.size(1)]
return self.dropout(x) # Instantiate Positional Encoding pos_encoder =
PositionalEncoding(d_model, max_seq_len) # Apply positional encoding to
the embedded input embedded_with_pos = pos_encoder(embedded_input)
print(f"Shape of embedded input with positional encoding:
{embedded_with_pos.shape}")
```

Code Cell 13

```
print(embedded_with_pos[0][1], embedded_with_pos.shape) #PE for anugrah
(1, batch, 10 total , 8 dimension)
```

Code Cell 14

```
print(f"Input Text: '{input_text}'") print(f"Input IDs:
{input_ids.squeeze(0)}") print("\nCombined Embeddings (Word Embedding +
Positional Encoding) for each token:") for i, token_id in
enumerate(input_ids.squeeze(0)): word = idx_to_word.get(token_id.item(),
'<unknown>') combined_embedding_vector = embedded_with_pos.squeeze(0)[i]
print(f"Token: '{word}' (ID: {token_id.item()}) -> Combined Embedding:
{combined_embedding_vector.tolist()}")
```

Code Cell 15

```
token_type_embedding = nn.Embedding(2, d_model) # Assuming 2 types: 0 for
sentence A, 1 for sentence B ''' Segment Embedding Tells the model which
sentence a token belongs to. Example: Sentence A: I love AI Sentence B:
It is powerful BERT input: [CLS] I love AI [SEP] It is powerful [SEP] '''
# Get segment embeddings for our token_type_ids segment_embeddings =
token_type_embedding(token_type_ids_new) # Add the segment embeddings to
the combined word and positional embeddings final_transformer_input =
embedded_with_pos + segment_embeddings print(f"Shape of token_type_ids:
{token_type_ids_new.shape}") print(f"Shape of segment embeddings:
{segment_embeddings.shape}") print(f"Shape of final input to Transformer:
{final_transformer_input.shape}") print("\nSegment Embeddings:")
print(segment_embeddings)
```

Code Cell 16

```
print('final transformer input') print(final_transformer_input)
```

Code Cell 17

```
import torch import torch.nn as nn import math class
TransformerEncoderLayer(nn.Module): def __init__(self, d_model,
num_heads, ffn_hidden_dim, dropout_rate=0.1): super().__init__() '''
d_model = 512 num_heads = 8 head_dim = 64 512 dimensions ■■ Head 1 : 64
■■ Head 2 : 64 ■■ Head 3 : 64 ■■ Head 4 : 64 ■■ Head 5 : 64 ■■ Head 6 :
64 ■■ Head 7 : 64 ■■ Head 8 : 64 ''' assert d_model % num_heads == 0 #if
it is divisible then return true and allow run self.d_model = d_model
self.num_heads = num_heads self.head_dim = d_model // num_heads #
Multi-Head Attention self.Wq = nn.Linear(d_model, d_model) self.Wk =
nn.Linear(d_model, d_model) self.Wv = nn.Linear(d_model, d_model) self.Wo
= nn.Linear(d_model, d_model) self.attn_dropout =
nn.Dropout(dropout_rate) # LayerNorm after MHA residual self.layer_norm1
= nn.LayerNorm(d_model) # Feed Forward Network self.ffn = nn.Sequential(
nn.Linear(d_model, ffn_hidden_dim), nn.ReLU(), nn.Linear(ffn_hidden_dim,
d_model) ) self.ffn_dropout = nn.Dropout(dropout_rate) # LayerNorm after
FFN residual self.layer_norm2 = nn.LayerNorm(d_model) def
_calculate_attention(self, Q, K, V, mask=None): scores = torch.ma
```

Code Cell 18

```

class TransformerDecoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, ffn_hidden_dim, dropout_rate=0.1):
        super().__init__()
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads
        ''' Target Tokens ↓ Embedding + Positional Encoding ↓ Masked Self-Attention ↓ Add & Norm ↓ Encoder-Decoder Attention ↓ Add & Norm ↓ Feed Forward Network ↓ Add & Norm ↓ Output '''
        # Masked Multi-Head Attention (Self-Attention for Decoder)
        self.masked_Wq = nn.Linear(d_model, d_model)
        self.masked_Wk = nn.Linear(d_model, d_model)
        self.masked_Wv = nn.Linear(d_model, d_model)
        self.masked_Wo = nn.Linear(d_model, d_model)
        self.masked_attn_dropout = nn.Dropout(dropout_rate)
        self.layer_norm1 = nn.LayerNorm(d_model)
        # Multi-Head Attention (Encoder-Decoder Attention)
        self.enc_dec_Wq = nn.Linear(d_model, d_model)
        self.enc_dec_Wk = nn.Linear(d_model, d_model)
        self.enc_dec_Wv = nn.Linear(d_model, d_model)
        self.enc_dec_Wo = nn.Linear(d_model, d_model)
        self.enc_dec_attn_dropout = nn.Dropout(dropout_rate)

```

Code Cell 19

```

# Re-defining TransformerEncoder for robustness
class TransformerEncoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads, ffn_hidden_dim, num_layers, max_seq_len, dropout_rate=0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_seq_len)
        self.encoder_layers = nn.ModuleList([TransformerEncoderLayer(d_model, num_heads, ffn_hidden_dim, dropout_rate) for _ in range(num_layers)])
    def forward(self, src, src_mask):
        # Embeddings + Positional Encoding
        x = self.embedding(src)
        x = self.positional_encoding(x)
        # Pass through encoder layers
        for layer in self.encoder_layers:
            x = layer(x, mask=src_mask)
        return x
# Re-defining FullTransformer for robustness
class FullTransformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model, num_heads, ffn_hidden_dim, num_encoder_layers, num_decoder_layers, max_seq_len, dropout_rate=0.1):
        super().__init__()
        self.encoder = TransformerEncoder(src_vocab_size, d_model, num_heads, ffn_hidden_dim, num_encoder_layers, max_seq_len, dropout_rate)
        self.decoder = TransformerDecoder(tgt_vocab_size, d_model, num_heads, ffn_hidden_dim, num_decoder_layers, max_seq_len, dropout_rate)
        self.output_linear = nn.Linear(d_model, tgt_vocab_size)

```